Aula 8

Usei as tabelas do exemplo dos slides: cliente(id_cliente, nome, email, telefone, ativo), produto(id_produto, nome, categoria, preco, estoque), pedido(id_pedido, criado_em, status, id_cliente), item_pedido(id_pedido, id_produto, quantidade, preco_praticado) e empregado(id_empregado, nome, id_supervisor).

Exercício 1 – INNER JOIN e autorrelacionamento

1.1 Pedidos com nome do cliente e status

```
SELECT p.id_pedido, c.nome AS cliente, p.status
FROM pedido AS p
INNER JOIN cliente AS c ON c.id_cliente = p.id_cliente
ORDER BY p.id_pedido;
```

1.2 Produtos e subtotal de cada item

```
SELECT i.id_pedido, pr.nome AS produto, i.quantidade, i.preco_praticado,
    i.quantidade * i.preco_praticado AS subtotal
FROM item_pedido AS i
INNER JOIN produto AS pr ON pr.id_produto = i.id_produto
ORDER BY i.id_pedido, pr.nome;
```

1.3 Caminho PEDIDO, ITEM_PEDIDO e PRODUTO

```
SELECT p.id_pedido, p.criado_em, pr.nome AS produto, i.quantidade
FROM pedido AS p
INNER JOIN item_pedido AS i ON i.id_pedido = p.id_pedido
INNER JOIN produto AS pr ON pr.id_produto = i.id_produto
ORDER BY p.id_pedido, pr.nome;
```

O item_pedido fica no meio porque é ele que liga pedido e produto (relação N:N).

1.4 Empregados e seus supervisores

```
SELECT e.nome AS empregado, s.nome AS supervisor
FROM empregado AS e
INNER JOIN empregado AS s ON s.id_empregado = e.id_supervisor;
```

A mesma tabela aparece duas vezes, por isso uso os apelidos: e é o empregado e s é o supervisor.

1.5 Por que cada ON está certo?

1.1: o pedido guarda o id_cliente, então ligo p.id_cliente com c.id_cliente. Cada pedido é de um cliente só.

1.2: o item guarda o id_produto, então ligo com o id do produto. Cada item é de um produto só.

1.3: primeiro ligo pedido com item pelo id_pedido, depois item com produto pelo id_produto. Se eu errasse uma dessas ligações, as linhas se multiplicariam sem sentido.

1.4: o id_supervisor do empregado tem que ser igual ao id_empregado do supervisor. Se eu ligasse id_empregado com id_empregado, cada pessoa viraria supervisora dela mesma. Como é INNER JOIN, quem não tem supervisor fica de fora.

Exercício 2 – Junções externas e ausência de correspondência

2.1 Todos os clientes, inclusive quem não tem pedido

```
SELECT c.nome, p.id_pedido
FROM cliente AS c
LEFT JOIN pedido AS p ON p.id_cliente = c.id_cliente
ORDER BY c.nome;
```

O cliente sem pedido aparece com id_pedido NULL.

2.2 Produtos nunca vendidos

```
SELECT pr.id_produto, pr.nome
FROM produto AS pr
LEFT JOIN item_pedido AS i ON i.id_produto = pr.id_produto
WHERE i.id_produto IS NULL
ORDER BY pr.nome;
```

O LEFT JOIN mantém todos os produtos. Os que nunca foram vendidos ficam com NULL do lado do item, e o WHERE pega só esses.

2.3 RIGHT JOIN reescrito com LEFT JOIN

```
-- com RIGHT JOIN
SELECT c.nome, p.id_pedido
FROM pedido AS p
RIGHT JOIN cliente AS c ON c.id_cliente = p.id_cliente;
-- com LEFT JOIN (mesmo resultado)
SELECT c.nome, p.id_pedido
FROM cliente AS c
LEFT JOIN pedido AS p ON p.id_cliente = c.id_cliente;
```

É só trocar as tabelas de lado: a tabela que eu quero preservar (cliente) vai pra esquerda.

2.4 Junção externa completa (FULL JOIN) simulada

```
SELECT c.nome, p.id_pedido
FROM cliente AS c
LEFT JOIN pedido AS p ON p.id_cliente = c.id_cliente
UNION
SELECT c.nome, p.id_pedido
FROM cliente AS c
RIGHT JOIN pedido AS p ON p.id_cliente = c.id_cliente;
```

O MySQL não tem FULL JOIN. Então junto um LEFT JOIN com um RIGHT JOIN usando UNION, que ainda tira as linhas repetidas.

2.5 Por que o filtro no WHERE pode eliminar linhas preservadas?

O LEFT JOIN cria linhas com NULL pros clientes sem pedido. Se eu coloco no WHERE algo como p.status = 'PAGO', o NULL nunca é igual a 'PAGO', então essas linhas somem e o LEFT JOIN vira na prática um INNER JOIN.

Pra manter os clientes sem pedido pago, o filtro vai no ON:

```
SELECT c.nome, p.id_pedido
FROM cliente AS c
```

LEFT JOIN pedido AS p ON p.id_cliente = c.id_cliente AND p.status = 'PAGO';

Exercício 3 – Agregações com GROUP BY e HAVING

3.1 Quantidade, menor, maior e média de preço por categoria

```
SELECT categoria, COUNT(*) AS quantidade,
    MIN(preco) AS menor_preco, MAX(preco) AS maior_preco,
    ROUND(AVG(preco), 2) AS preco_medio
FROM produto
GROUP BY categoria
ORDER BY categoria;
```

3.2 Pedidos por cliente, incluindo zero

```
SELECT c.id_cliente, c.nome, COUNT(p.id_pedido) AS total_pedidos
FROM cliente AS c
LEFT JOIN pedido AS p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome
ORDER BY total_pedidos DESC, c.nome;
```

Tem que ser COUNT(p.id_pedido) e não COUNT(*). O COUNT(*) conta a linha do cliente sem pedido como 1, e o COUNT(p.id_pedido) ignora o NULL e dá 0.

3.3 Clientes com pelo menos dois pedidos

```
SELECT c.id_cliente, c.nome, COUNT(p.id_pedido) AS total_pedidos
FROM cliente AS c
INNER JOIN pedido AS p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) >= 2
ORDER BY total_pedidos DESC, c.nome;
```

3.4 Valor total de cada pedido

```
SELECT i.id_pedido, SUM(i.quantidade * i.preco_praticado) AS valor_total
FROM item_pedido AS i
GROUP BY i.id_pedido
ORDER BY i.id_pedido;
```

3.5 Por que usei WHERE, GROUP BY e HAVING?

GROUP BY: junta as linhas em grupos (por categoria, por cliente, por pedido), senão o COUNT, SUM e AVG calculariam a tabela inteira de uma vez.
HAVING: filtra depois que os grupos foram feitos. No 3.3 preciso dele porque o COUNT só existe depois de agrupar.
WHERE: filtra as linhas antes de agrupar. Nesses exercícios não precisei dele, mas se quisesse só produtos com estoque, por exemplo, ele iria antes do GROUP BY.

Desafio – 8 consultas multitabelas e indicadores

Q1. INNER JOIN (pedido, cliente)

Pergunta: quais pedidos foram feitos por cada cliente e em que data?

```
SELECT p.id_pedido, c.nome AS cliente, p.criado_em, p.status
```

FROM pedido AS p
INNER JOIN cliente AS c ON c.id_cliente = p.id_cliente
ORDER BY p.criado_em DESC;
Resultado: uma linha por pedido, com o nome de quem comprou.
Interpretação: cada pedido tem um cliente só, então não repete linha. Cliente sem pedido não aparece.

Q2. INNER JOIN (item, produto)
Pergunta: quanto foi vendido de cada produto em cada pedido?
SELECT i.id_pedido, pr.nome AS produto, i.quantidade,
    i.quantidade * i.preco_praticado AS subtotal
FROM item_pedido AS i
INNER JOIN produto AS pr ON pr.id_produto = i.id_produto
ORDER BY i.id_pedido, pr.nome;
Resultado: uma linha por item, com o subtotal.
Interpretação: uso o preco_praticado (o preço da venda) e não o preço atual do produto, porque o preço pode ter mudado depois.

Q3. Junção externa (clientes sem pedido)
Pergunta: quais clientes ainda não fizeram nenhum pedido?
SELECT c.id_cliente, c.nome, c.email
FROM cliente AS c
LEFT JOIN pedido AS p ON p.id_cliente = c.id_cliente
WHERE p.id_pedido IS NULL
ORDER BY c.nome;
Resultado: só os clientes sem pedido.
Interpretação: dá pra usar numa campanha pra quem nunca comprou. Se todos já compraram, volta vazio.

Q4. Junção externa (busca por ausência)
Pergunta: quais produtos nunca foram vendidos?
SELECT pr.id_produto, pr.nome, pr.estoque
FROM produto AS pr
LEFT JOIN item_pedido AS i ON i.id_produto = pr.id_produto
WHERE i.id_produto IS NULL
ORDER BY pr.nome;
Resultado: os produtos que não aparecem em nenhum item.
Interpretação: o filtro do WHERE usa a coluna da chave do item (i.id_produto), que só é NULL quando não achou correspondência. Produto parado com estoque alto é um bom candidato a promoção.

Q5. Autorrelacionamento
Pergunta: quem é o supervisor de cada empregado, incluindo quem não tem?
SELECT e.nome AS empregado, s.nome AS supervisor
FROM empregado AS e
LEFT JOIN empregado AS s ON s.id_empregado = e.id_supervisor
ORDER BY e.nome;
Resultado: uma linha por empregado, e o supervisor fica NULL pra quem está no topo.

Interpretação: uso LEFT JOIN pra não perder o chefe, que não tem supervisor. Com INNER JOIN ele sumiria da lista.

Q6. Agregação com LEFT JOIN (pedidos por cliente)
Pergunta: quantos pedidos cada cliente fez, contando zero?

```
SELECT c.id_cliente, c.nome, COUNT(p.id_pedido) AS total_pedidos
FROM cliente AS c
LEFT JOIN pedido AS p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome
ORDER BY total_pedidos DESC, c.nome;
```

Resultado: uma linha por cliente, e quem nunca comprou aparece com 0.
Interpretação: o COUNT(p.id_pedido) ignora o NULL e dá o zero certo. Com COUNT(*) esse cliente sairia com 1, que estaria errado.

Q7. Agregação com LEFT JOIN (produtos vendidos)
Pergunta: quantas unidades de cada produto foram vendidas, contando zero?

```
SELECT pr.id_produto, pr.nome, COALESCE(SUM(i.quantidade), 0) AS unidades_vendidas
FROM produto AS pr
LEFT JOIN item_pedido AS i ON i.id_produto = pr.id_produto
GROUP BY pr.id_produto, pr.nome
ORDER BY unidades_vendidas DESC, pr.nome;
```

Resultado: uma linha por produto, e os nunca vendidos aparecem com 0.
Interpretação: o SUM de só NULL dá NULL, então uso o COALESCE pra trocar por 0. No topo ficam os mais vendidos.

Q8. WHERE e HAVING juntos
Pergunta: quais categorias com produtos em estoque têm pelo menos 2 produtos, e qual a média de preço delas?

```
SELECT categoria, COUNT(*) AS quantidade, ROUND(AVG(preco), 2) AS preco_medio
FROM produto
WHERE estoque > 0
GROUP BY categoria
HAVING COUNT(*) >= 2
ORDER BY preco_medio DESC;
```

Resultado: uma linha por categoria que passou nos dois filtros.
Interpretação: o WHERE tira os produtos sem estoque antes de agrupar, e o HAVING tira as categorias com menos de 2 produtos depois de agrupar. Se nenhuma categoria passar, volta vazio.